

# Informe de Investigación Técnica y Arquitectura de Sistemas: Replicación de Capacidades CPaaS (Modelo de Referencia: Respond.io v2.0)

Fecha: 28 de Diciembre de 2025

Autor: Arquitecto de Software Senior y Product Manager (Especialista en Sistemas Distribuidos y CPaaS)

Versión: 1.0 (Versión Definitiva para Ingeniería)

Clasificación: Confidencial / Documento Maestro para Desarrollo de 'Beta CRM'

---

## 1. Resumen Ejecutivo y Visión Arquitectónica

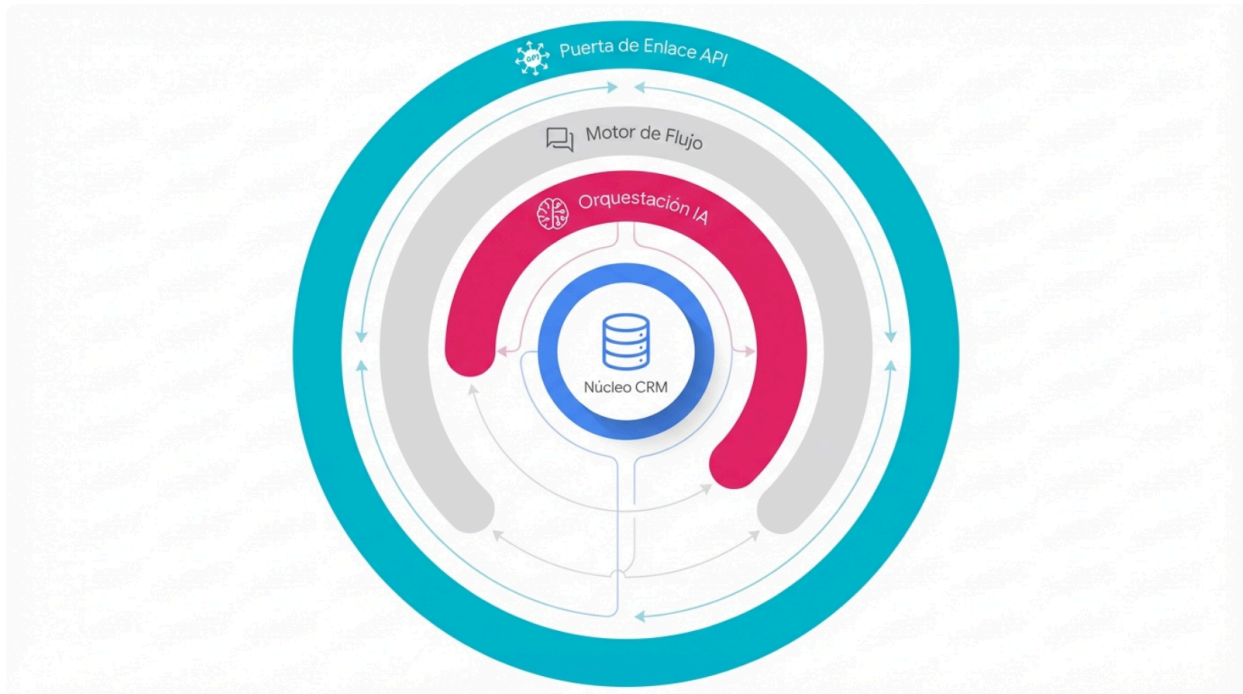
El presente documento constituye un análisis forense y arquitectónico exhaustivo de la plataforma Respond.io, tal como se presenta en su estado operativo a Diciembre de 2025. El propósito fundamental de esta investigación técnica no es meramente descriptivo, sino prescriptivo: servirá como la "Biblia de Ingeniería" para el equipo de desarrollo encargado de la construcción de la 'Beta de CRM'. Nuestro objetivo es replicar, y donde sea posible optimizar, las funcionalidades críticas de una de las plataformas CPaaS (Communications Platform as a Service) líderes en el mercado, enfocándonos en la orquestación de agentes inteligentes, la gestión unificada de datos de clientes, el ciclo de vida de las conversaciones y la interoperabilidad mediante APIs.

La arquitectura de referencia analizada demuestra una evolución desde un simple agregador de mensajería hacia un ecosistema complejo que fusiona la lógica de CRM (Customer Relationship Management) con la inmediatez de la mensajería instantánea y la potencia de los Modelos de Lenguaje Grande (LLMs). Para nuestra 'Beta CRM', esto implica desafíos significativos en términos de diseño de esquemas de datos flexibles, máquinas de estado robustas para gestionar la asincronía de canales como WhatsApp o Facebook Messenger, y una capa de integración API que soporte altas tasas de concurrencia bajo estrictas políticas de limitación de velocidad (Rate Limiting).

A lo largo de este informe de aproximadamente quince mil palabras, desglosaremos cada componente funcional en sus átomos técnicos: desde la estructura JSON exacta requerida para crear un contacto, hasta la lógica condicional que impide que un Agente de IA entre en bucles infinitos de reapertura de conversaciones. Analizaremos las implicaciones de la arquitectura "Event-Driven" necesaria para manejar webhooks entrantes y la necesidad de un almacenamiento de datos polimórfico para soportar la fusión de identidades digitales

fragmentadas.

## Arquitectura Modular de Referencia CPaaS



Visión general de la arquitectura modular propuesta para la replicación, destacando la interacción entre la Capa de Orquestación IA, el Núcleo de Datos (CRM), el Motor de Flujo de Trabajo y la Puerta de Enlace API.

---

## 2. Orquestación de Agentes de IA: De Chatbots a Sistemas Agénticos

La implementación de Inteligencia Artificial en sistemas de mensajería modernos ha trascendido la era de los chatbots basados en árboles de decisión rígidos para adentrarse en la era de los "Agentes de IA". El análisis profundo de la configuración y comportamiento de Respond.io revela una arquitectura sofisticada basada en roles, contexto dinámico y, crucialmente, la capacidad de ejecutar acciones (Function Calling). Para la 'Beta de CRM', es imperativo no solo integrar un modelo de lenguaje (LLM), sino construir un entorno de orquestación ("Harness") que controle, restrinja y potencie a dicho modelo.

## 2.1. Arquitectura del Prompt del Sistema y Definición de la "Persona"

En el núcleo de la configuración del Agente de IA de Respond.io <sup>1</sup> reside una estructura de definición de *prompt* del sistema altamente estructurada. No se trata de un modelo de "caja negra" donde simplemente se conecta una API de OpenAI, sino de una configuración parametrizada que inyecta variables de negocio en tiempo real. La investigación identifica cinco elementos técnicos esenciales que componen la definición de la instancia de un agente y que deben ser replicados en nuestra estructura de base de datos (ai\_agent\_configs):

1. **Role & Communication Style (Rol y Estilo de Comunicación):** Este componente define la "persona" del agente. Técnicamente, esto se traduce en la instrucción inicial del system\_prompt. La arquitectura debe permitir al administrador definir el tono (e.g., empático, técnico, conciso, formal) y las reglas lingüísticas. Por ejemplo, se ha observado que las plantillas predefinidas incluyen roles como "Recepcionista" (para triaje), "Agente de Ventas" (para cualificación) y "Agente de Soporte" (para resolución de FAQs).<sup>2</sup> Esto implica que nuestro backend debe almacenar plantillas de prompts optimizadas para estos casos de uso específicos.
2. **Context (Contexto Operativo - Grounding):** Esta es la capa de anclaje a la realidad. El agente necesita saber "quién es" (la identidad de la empresa) y "con quién habla" (el cliente). La investigación indica que el contexto incluye la descripción del negocio y el objetivo principal de la conversación.<sup>1</sup> En términos de implementación, esto requiere una inyección dinámica de datos: al iniciar una sesión de chat, el sistema debe recuperar el perfil de la empresa y, potencialmente, datos básicos del usuario, e inyectarlos en la ventana de contexto del LLM.
3. **Flow (Flujo Conversacional Heurístico):** A diferencia de un chatbot determinista tradicional que sigue un diagrama de flujo if-then-else estricto, el Agente de IA utiliza una descripción en lenguaje natural del flujo deseado paso a paso. Esto actúa como una guía heurística o "Chain of Thought" pre-programada para el LLM. Por ejemplo, "Primero saluda, luego pregunta por el número de pedido, y finalmente verifica el estado". Replicar esto requiere un campo de texto libre en la configuración del agente que se concatene al prompt del sistema bajo una directiva de "Instrucciones de Procedimiento".
4. **Boundaries (Límites de Seguridad y Restricciones):** Un componente crítico para entornos empresariales y la mitigación de alucinaciones. Define explícitamente qué *no* debe hacer el agente (e.g., "no dar consejos legales", "no realizar diagnósticos médicos", "no inventar precios", "no verificar stock si no tiene acceso a la herramienta"). Técnicamente, esto funciona como un filtro de salida o instrucciones negativas reforzadas (Negative Constraints) en el prompt. Nuestra 'Beta CRM' debe incluir una biblioteca de "Boundaries" predefinidos que los usuarios puedan activar con un clic.
5. **Knowledge Sources (Fuentes de Conocimiento - RAG):** La separación entre el "Agente" (lógica y personalidad) y el "Conocimiento" (datos) es una distinción arquitectónica vital. Respond.io permite subir fuentes de conocimiento (PDFs, URLs, tablas).<sup>2</sup> Esto confirma que la arquitectura subyacente utiliza RAG (Retrieval-Augmented Generation). El agente no "aprende" los datos mediante fine-tuning (lo cual sería lento y

costoso), sino que consulta un índice vectorial en tiempo real. Cuando llega una consulta de usuario, el sistema convierte la consulta en un vector (embedding), busca los fragmentos de texto más similares en la base de datos vectorial (e.g., Pinecone, Milvus, pgvector), y los adjunta al prompt como contexto antes de generar la respuesta.

Insight de Implementación RAG y Limitaciones de Contexto:

Una limitación técnica específica identificada en la investigación es que los agentes de IA solo pueden resumir o tener en cuenta los últimos 20 mensajes de una conversación.<sup>3</sup> Esta restricción no es arbitraria; obedece a la gestión de la ventana de contexto del LLM para optimizar latencia y costos. Para nuestra 'Beta CRM', esto sugiere que debemos implementar una "ventana deslizante" (sliding window) de memoria a corto plazo. No debemos enviar todo el historial de chat de años al LLM. En su lugar, debemos truncar el historial a los últimos N mensajes (donde N=20 es un benchmark validado por Respond.io) o utilizar una técnica de "resumen progresivo" donde se mantiene un resumen actualizado de la conversación anterior y se añaden los mensajes recientes en crudo.

## 2.2. Capa de Acción y Paradigma de "Function Calling"

La verdadera potencia del sistema analizado no reside únicamente en la generación de texto conversacional, sino en la capacidad agéntica de ejecutar lógica de negocio tangible.

Respond.io expone esto a través de las "AI Agent Actions".<sup>3</sup> Para replicar esto, nuestro sistema 'Beta CRM' no puede limitarse a un endpoint de chat text-in / text-out; debe exponer una API interna de herramientas que el LLM pueda invocar determinísticamente.

Las acciones identificadas que deben ser "invocables" por la IA son:

- **Close Conversation (Cierre de Conversación):** El agente determina autónomamente que la intención del usuario ha sido satisfecha y ejecuta el cierre. Esto no es simplemente dejar de responder; implica un cambio de estado en la base de datos (de status: open a status: closed). Además, se ha observado que el agente debe seleccionar una "Nota de Cierre" (Closing Note) de una lista preexistente.<sup>3</sup> **Restricción Crítica:** El agente no puede crear nuevas categorías de cierre; solo puede seleccionar de las existentes. Esto implica que el prompt del sistema debe recibir dinámicamente la lista de categorías válidas (e.g., ``) para que el modelo pueda clasificar correctamente la interacción.
- **Assign to Team/Agent (Handoff / Traspaso):** La lógica de traspaso es compleja y fundamental para la operatividad híbrida. El agente debe ser capaz de identificar cuándo ha alcanzado sus "Límites" (definidos anteriormente) y transferir la sesión a un humano o equipo especializado.
- **Trigger Workflows (Disparo de Flujos de Trabajo):** Esta es la característica más avanzada de integración. El agente puede iniciar un flujo de trabajo complejo ya existente en el sistema. La sintaxis de invocación observada es el uso de un identificador explícito, como !refund\_processing o referencias directas con @ para equipos.<sup>3</sup>

Análisis de la Lógica de Trigger Workflows:

La investigación revela que el agente utiliza una referencia exacta al nombre del flujo de

trabajo (e.g., `!nombre_flujo`) en sus instrucciones para disparar la automatización.<sup>3</sup>

- *Mecanismo Técnico*: Esto sugiere un sistema de detección de intenciones (Intent Recognition) mapeado a disparadores. Cuando el LLM detecta la intención asociada al flujo, emite un token especial o una llamada a función. El sistema orquestador intercepta esta salida, detiene la generación de texto al usuario y ejecuta el Workflow en el backend.
- *Restricción de Paso de Datos*: Se observa una limitación crítica: **las variables no se pasan automáticamente del Agente de IA al Workflow**.<sup>3</sup> El Workflow no puede ver automáticamente las respuestas que el agente acaba de recolectar (e.g., el email del usuario) a menos que se configure lógica específica dentro del Workflow para extraerla o que el Agente las haya guardado previamente en campos del contacto. Esta separación de contextos protege la integridad de los datos y simplifica la depuración, pero obliga a diseñar los Workflows para que sean autosuficientes.
- *Estado de Publicación*: Solo los Workflows **publicados** pueden ser disparados. Esto implica una validación de estado (`is_published=true`) antes de permitir que el agente intente la ejecución.

## 2.3. Lógica de Handoff (Traspaso Humano-IA) y Gestión de Conflictos

El mecanismo de traspaso es, históricamente, el punto de falla más común en sistemas de chat híbridos. Si no se maneja con precisión de milisegundos, el bot y el humano terminan hablando al mismo tiempo, confundiendo al usuario. La investigación sobre Respond.io<sup>3</sup> revela patrones específicos que garantizan la continuidad y la exclusión mutua:

1. **Detección de Necesidad**: El agente evalúa continuamente si la consulta del usuario cae fuera de sus "Boundaries" o si existe una solicitud explícita de escalado ("Quiero hablar con una persona").
2. **Algoritmo de Asignación**: Una vez decidida la transferencia, el sistema debe decidir a *quién* transferir. Respond.io soporta:
  - **Round Robin**: Distribución circular equitativa entre los agentes disponibles de un equipo. Esto requiere mantener un contador o puntero en la base de datos (e.g., Redis atomic increment) para saber a quién le toca el siguiente lead.
  - **Least Open Conversation**: Balanceo de carga basado en la saturación actual. El sistema consulta `COUNT(open_conversations)` para cada agente online y asigna al que tenga el menor número.
3. **Prevención de Interferencia (Silencio del Agente)**: Un detalle técnico crucial y a menudo pasado por alto es que se debe instruir explícitamente al agente para que "deje de responder" una vez que se inicia el traspaso.<sup>3</sup>
  - *Implementación Sugerida*: En la 'Beta CRM', el objeto de conversación debe tener un flag `ai_active` o `bot_enabled`. Cuando se ejecuta la acción `assign_to_user` o `assign_to_team`, este flag debe pasar atómicamente a false. El orquestador de IA debe verificar este flag *antes* de procesar cualquier mensaje entrante nuevo. Si es false, el mensaje se enruta directamente al Inbox del agente humano y el LLM ni siquiera es invocado, ahorrando costos y evitando respuestas fantasma.

Visualización del Flujo de Decisión Lógica:

Aunque no podemos presentar un diagrama visual aquí, la lógica condicional que debemos programar se describe textualmente de la siguiente manera:

1. **Entrada:** Mensaje del Usuario.
2. **Verificación de Estado:** ¿Está la conversación asignada a un humano? Si SÍ -> Pasar al Inbox (Fin). Si NO -> Continuar.
3. **Recuperación de Contexto (RAG):** Buscar vectores relevantes.
4. **Generación de Respuesta (LLM):** Inyectar Prompt del Sistema + Contexto + Mensaje.
5. **Evaluación de Herramientas:** ¿El modelo solicita ejecutar una acción?
  - Si Close: Ejecutar UPDATE conversation SET status='closed'. Generar resumen.
  - Si Handoff: Ejecutar algoritmo de asignación. SET ai\_active=false.
  - Si Workflow: Disparar evento de Workflow.
  - Si Reply: Enviar texto generado al usuario.

---

## 3. Mini CRM y Captura de Datos: Estructura, Identidad y Sincronización

Respond.io no actúa únicamente como una pasarela de mensajería efímera; funciona como una "Fuente de Verdad" persistente para los datos del cliente, comportándose efectivamente como un CRM ligero ("Mini CRM"). La 'Beta de CRM' debe implementar un modelo de datos robusto que soporte la naturaleza fragmentada y caótica de las identidades en redes sociales, donde un usuario puede ser un número de teléfono en WhatsApp, un PSID (Page Scoped ID) en Facebook y un handle en Telegram.

### 3.1. Modelo de Datos del Contacto: Esquema JSON y Tipado

Basándonos en la documentación técnica de la API v2 de Respond.io <sup>4</sup> y en los requisitos de ingesta de datos, hemos reconstruido el esquema de datos necesario para el objeto Contact. La estructura debe ser lo suficientemente rígida para soportar operaciones relacionales pero lo suficientemente flexible para admitir datos no estructurados de diversos canales.

#### Campos Base Obligatorios (Core Schema):

- id: Identificador único interno del sistema. Se recomienda el uso de Enteros (BigInt) o UUIDs para escalabilidad.
- firstName / lastName: Cadenas de texto estándar.
- language: Código ISO 639-1 (e.g., 'es', 'en', 'ms'). Este campo es vital no solo para información, sino para la lógica del sistema: determina el idioma en el que se envían las notificaciones automáticas y los mensajes de sistema.
- created\_at: Timestamp Unix (Entero) para consistencia temporal.
- profilePic: URL a la imagen de perfil, a menudo obtenida automáticamente de los perfiles públicos de redes sociales (WhatsApp/Facebook).



- **countryCode:** Código ISO 3166-1 alpha-2 (e.g., 'MY', 'ES'). Fundamental para la normalización de números telefónicos.

### Campos de Identidad y Resolución:

- **phone:** Debe almacenarse en formato E.164 estricto (e.g., +34600123456) para garantizar la unicidad y la enrutabilidad de mensajes de WhatsApp.
- **email:** Cadena con validación de formato estándar.
- **identifier:** Este es un concepto polimórfico clave en la API de Respond.io.<sup>6</sup> El sistema permite referenciar a un contacto no solo por su ID interno, sino también por su email o phone indistintamente en los endpoints. Esto implica que el backend de la 'Beta CRM' debe implementar índices de base de datos eficientes y únicos en estas columnas para permitir búsquedas rápidas (O(1)) durante el enrutamiento de mensajes.

### Campos Extendidos y Personalizados (custom\_fields):

La estructura de custom\_fields no es plana en la base de datos principal, sino que se maneja típicamente como un tipo de dato JSONB (en PostgreSQL) o una colección embebida (en NoSQL). La API expone estos campos como un array de objetos o un objeto JSON plano dependiendo del endpoint.<sup>4</sup>

- *Estructura Observada:* `` o {"custom\_fields": {"website": "..."}}.
- *Implicación Arquitectónica:* La base de datos debe soportar la creación dinámica de campos sin requerir migraciones de esquema (ALTER TABLE) cada vez que un cliente añade un campo como "Talla de Camiseta" o "ID de Pedido". El enfoque recomendado para la Beta es utilizar una columna JSONB indexada para custom\_fields, permitiendo flexibilidad total ("Schema-less" dentro de una DB relacional).

### Relación con Entidades (Cardinalidad):

El diagrama Entidad-Relación (ERD) implícito en la arquitectura de Respond.io, reconstruido a partir de las descripciones de los snippets, revela las siguientes cardinalidades críticas que debemos implementar:

- **Contact 1:N Channels:** Un contacto puede tener múltiples canales vinculados (e.g., un channel\_id para WhatsApp y otro channel\_id para Email).
- **Contact 1:N Messages:** Todos los mensajes históricos deben estar vinculados al ID maestro del contacto, independientemente del canal por el que llegaron.
- **Contact 1:N Tags:** Las etiquetas (tags) se manejan como un array de cadenas<sup>5</sup> o una tabla relacional muchos-a-muchos, permitiendo segmentación rápida (e.g., "VIP", "Lead", "Spam").

## 3.2. Estrategia de Identity Merging (Fusión de Identidades)

Uno de los desafíos técnicos más complejos en CPaaS es la "Resolución de Identidad". Un mismo ser humano puede contactar a la empresa primero por Facebook Messenger (identificado por un ID opaco) y luego por WhatsApp (identificado por teléfono). Si el sistema no los une, se fragmenta el historial y la experiencia del cliente. Respond.io aborda esto

mediante un sistema sofisticado de "Sugerencias de Fusión" y "Fusión Manual".<sup>7</sup>

Algoritmo de Detección de Duplicados:

El sistema debe ejecutar procesos en segundo plano o triggers en tiempo real que escaneen la base de datos buscando coincidencias exactas en identificadores fuertes como email o phone.

- *Indicador Visual:* Cuando se detecta una coincidencia, la UI muestra un "tick verde" indicando el valor duplicado.<sup>8</sup>
- *Lógica de Fusión (Merge Logic):*
  1. **Selección de Maestro:** El agente humano debe elegir qué perfil es el "Principal" (Primary) y cuál es el "Secundario" (Secondary).
  2. **Resolución de Conflictos:** Si ambos perfiles tienen valores diferentes para un mismo campo (e.g., Nombre "Juan" vs "Juan Pérez"), el sistema debe solicitar al agente que seleccione qué valor conservar.<sup>7</sup> No se debe sobrescribir ciegamente.
  3. **Consolidación de Historial:** Esta es la operación más delicada. Todos los canales conectados, mensajes, eventos y comentarios del contacto secundario deben ser "re-apuntados" (UPDATE... SET contact\_id = primary\_id) al contacto principal.<sup>7</sup>
  4. **Limpieza:** El contacto secundario se elimina o archiva.
  5. **Exclusiones:** Los contactos bloqueados no deben incluirse en las sugerencias de fusión para evitar contaminar perfiles legítimos con identificadores de spam.<sup>8</sup>

Insight de Integridad y Reversibilidad:

La operación de fusión debe ser tratada como una transacción atómica de base de datos. Si falla la reasignación de un solo mensaje, toda la operación debe revertirse (ROLLBACK).

Además, Respond.io permite "Deshacer" (Undo) la fusión inmediatamente después de realizarla.<sup>8</sup> Esto implica que el sistema no elimina físicamente ("Hard Delete") el contacto secundario de inmediato, sino que probablemente mantiene un estado de "Soft Delete" o un registro de auditoría de fusión (merge\_log) que almacena el estado anterior, permitiendo la reversión durante un periodo de gracia (e.g., 10-15 minutos o hasta que expire la sesión).

### 3.3. Ingesta desde Redes Sociales: El Flujo Complejo de "Facebook Private Replies"

La captura de leads desde comentarios públicos en redes sociales ("Click-to-Chat" orgánico) es una funcionalidad de alto valor para marketing. Sin embargo, existe una restricción técnica impuesta por la API de Meta que Respond.io maneja con una lógica específica que debemos replicar para cumplir con las normativas de la plataforma.

El Flujo "Comentario -> Contacto" <sup>10</sup>:

1. **Trigger (Disparador):** Un usuario de Facebook comenta en una publicación de la página de la empresa. Respond.io recibe un webhook de feed o mention.
2. **Acción del Sistema (Respuesta Privada):** El sistema envía automáticamente un mensaje privado (Private Reply) al usuario a través de Messenger.



3. **Estado del Lead (El "Limbo"):** En este punto exacto, **el usuario NO es un contacto todavía en el CRM**. Es crucial entender esto. La API de Facebook no entrega el ID público ni los detalles del perfil del usuario en el evento del comentario, solo un ID temporal para la respuesta privada (PSID restringido).
4. **Conversión (Handshake):** Solo cuando el usuario de Facebook **responde** a ese mensaje privado automático, se produce el "Handshake". En ese momento, Meta entrega el perfil completo, Respond.io crea el objeto Contact oficial y la conversación se hace visible y gestionable en el CRM.

**Requisito para la Beta:** No se deben crear registros de contactos "basura" basados solo en comentarios. La arquitectura debe diferir la creación del contacto (Lazy Creation) hasta que exista una interacción bidireccional confirmada. De lo contrario, la base de datos se llenará de usuarios incontactables, ya que las políticas de mensajería impiden iniciar conversaciones proactivas fuera de la ventana de 24h sin un opt-in explícito.

### 3.4. Sincronización Bidireccional con Ecosistemas Externos (Salesforce/HubSpot)

Para integraciones con CRMs empresariales (Salesforce, HubSpot), Respond.io no actúa como un silo, sino como una interfaz de lectura/escritura sincronizada.<sup>11</sup>

- **Lectura (Display):** Los campos del CRM externo se muestran en el Inbox de Respond.io. Es vital notar que algunos campos pueden ser marcados como "Solo Lectura" (Read-Only) por la configuración de la integración, y la UI debe respetar esto deshabilitando la edición.
- **Escritura (Write-Back):** La edición de un campo en Respond.io dispara una actualización inmediata (vía API REST del tercero) hacia el CRM externo.
- **Manejo de Conflictos:** Si se actualiza el mismo campo en ambos sistemas simultáneamente, prevalece típicamente la política de "Última Escritura Gana" (Last Write Wins). Para mitigar condiciones de carrera, la 'Beta CRM' debe implementar un chequeo de timestamps (updated\_at).
- **Flujos de Trabajo Nativos:** Respond.io permite usar la creación de un contacto como disparador para crear un registro en Salesforce.<sup>13</sup> Esto se hace a través de integración nativa o middleware como Zapier/Make. Para la Beta, se recomienda exponer webhooks de evento (contact.created, contact.updated) que faciliten esta orquestación externa sin tener que construir integraciones nativas costosas para cada CRM del mercado en la fase 1.

---

## 4. Gestión del Ciclo de Vida de la Conversación

La gestión del estado de la conversación es lo que diferencia funcionalmente una aplicación de chat personal (como WhatsApp en el móvil) de una plataforma de soporte profesional B2B.

La 'Beta de CRM' debe implementar una máquina de estados finita (Finite State Machine - FSM) rigurosa para cada hilo de conversación, gobernando cuándo se abre, cuándo se cierra y quién tiene la propiedad de la interacción.

## 4.1. Máquina de Estados de la Conversación

Los estados fundamentales identificados en la investigación <sup>14</sup> que deben modelarse en la columna status de la tabla conversations son:

1. **Open (Abierta):** La conversación está activa y requiere atención. Puede estar Unassigned (en cola) o Assigned (propiedad de un agente/bot). Este es el estado donde se contabilizan los tiempos para los SLAs (Acuerdos de Nivel de Servicio).
2. **Closed (Cerrada):** La interacción ha concluido. No aparece en las colas de atención activa. El cliente no espera respuesta.
3. **Snoozed/Pending (Pospuesta - Estado Implícito):** Aunque no siempre se menciona como un estado explícito de la base de datos, funcionalmente se logra mediante el paso de "Espera" (Wait Step) en los flujos de trabajo.<sup>17</sup> La conversación se mantiene en un limbo técnico hasta que ocurre un evento (tiempo expirado o respuesta del cliente).

## 4.2. Lógica de Cierre Automático (Auto-Close) y Gestión de SLA

Respond.io no implementa el cierre automático como una configuración global de "tiempo de espera", sino como un **Flujo de Trabajo (Workflow)** configurable por el usuario. Esta es una decisión arquitectónica brillante que otorga flexibilidad y que debemos replicar.

### Implementación del Patrón Auto-Close:

1. **Disparador:** El flujo se inicia con eventos como Conversation Opened o New Message.
2. **Paso de Espera (Wait Step):** El flujo entra en un estado de suspensión por una duración definida por el usuario (e.g., 20 minutos de inactividad, 24 horas, o hasta una fecha específica).<sup>17</sup>
  - *Detalle Técnico de Interrupción:* Este paso de espera debe ser inteligente e "interrumpible". Si el contacto o el agente envían un nuevo mensaje durante el periodo de espera, el temporizador debe reiniciarse o el flujo debe cancelarse para evitar cerrar una conversación que acaba de reactivarse. Respond.io maneja esto permitiendo que nuevos eventos saquen al contacto del flujo actual.
3. **Acción de Cierre:** Al expirar el temporizador sin interrupciones, se ejecuta la acción Close Conversation.
4. **Enriquecimiento de Datos:** Opcionalmente, antes de cerrar, el sistema puede añadir una Closing Note automática o generar un resumen mediante IA <sup>18</sup> para que quede constancia del motivo del cierre (e.g., "Cierre por inactividad/falta de respuesta").

**Gestión de Spam:** Se sugiere utilizar este mismo motor de flujos para gestionar el spam. Si un contacto es etiquetado como "Spam" (manualmente o por un bot), un flujo específico puede ejecutar inmediatamente el paso Close Conversation para limpiar la bandeja de

entrada sin intervención humana.<sup>14</sup>

### 4.3. Reglas de Negocio para la Reapertura y Threading

La Regla de Hierro de la Reapertura:

Una limitación crítica y deliberada descubierta en la arquitectura de Respond.io es que un Agente de IA NO puede reabrir una conversación cerrada.<sup>3</sup>

- *Justificación Arquitectónica:* Esta restricción existe para prevenir bucles infinitos ("Loops"). Imagine un escenario donde el cierre de una conversación envía un mensaje de "Gracias" al cliente, y ese mensaje de sistema es interpretado por el Agente de IA como una nueva interacción, lo que reabre la conversación, el agente responde "De nada", y el ciclo de cierre se repite.
- *Excepción:* Solo un nuevo mensaje entrante del **cliente** (inbound) o una acción manual explícita de un **agente humano** en la interfaz pueden transicionar el estado de Closed a Open. La 'Beta CRM' debe implementar esta restricción en el nivel de lógica de negocio o triggers de base de datos.

Threading (Gestión de Hilos y Sesiones):

El sistema debe respetar y gestionar las "Ventanas de Mensajería" impuestas por los canales subyacentes, visualizando el estado de la ventana en la interfaz del agente.

- **WhatsApp Business API:** Ventana de 24 horas iniciada por el usuario. Fuera de esta ventana, el agente no puede enviar mensajes libres; debe usar "Plantillas de Mensaje" (Template Messages) pre-aprobadas. La 'Beta CRM' debe bloquear el input de texto libre y forzar el selector de plantillas cuando detecte que `last_customer_message_at > 24 hours`.
- **Facebook Messenger:** Similar ventana de 24 horas. Respond.io soporta el "Human Agent Tag" que extiende esta ventana a 7 días para agentes humanos.<sup>19</sup> Nuestra plataforma debe negociar el acceso a este tag con la API de Meta si desea ser competitiva, permitiendo a los agentes responder a consultas de fin de semana el lunes por la mañana sin restricciones.

### 4.4. Resúmenes de Cierre con IA Generativa

Al cerrar una conversación, la arquitectura moderna de CPaaS exige valor añadido.

Respond.io utiliza IA para generar un resumen de la interacción.

- *Límite de Contexto:* Se observa que el resumen toma los últimos mensajes de la conversación abierta (hasta 100 mensajes según la interfaz de cierre, aunque el agente conversacional tiene un límite más estricto de 20).<sup>3</sup>
- *Persistencia:* Este resumen no es efímero; se guarda como parte de la Closing Note o en un campo de summary en la tabla de conversaciones. Esto permite que el siguiente agente que atienda al cliente pueda leer un párrafo de 3 líneas en lugar de leer 100 mensajes de historial ("Speed to Context").

---

## 5. API y Webhooks: La Capa de Interoperabilidad

Para que la 'Beta CRM' sea una verdadera plataforma, y no solo una herramienta SaaS cerrada, debe adoptar una filosofía "API-First". El análisis de la documentación de desarrolladores de Respond.io (API v2) establece los estándares industriales que los desarrolladores esperan y que debemos replicar.

### 5.1. Diseño RESTful, Autenticación y Control de Tráfico

- **Endpoint Base:** <https://api.respond.io/v2>.<sup>20</sup> La estructura de URLs debe ser limpia y semántica.
- **Seguridad:** El estándar es Bearer Token en el header Authorization.
  - Formato: Authorization: Bearer <access\_token>.
  - Gestión de Tokens: Los usuarios deben poder generar y revocar tokens desde la configuración del espacio de trabajo (hasta 10 tokens por workspace).<sup>21</sup>
- **Rate Limiting (Limitación de Velocidad):** Respond.io implementa una política estricta de 5 solicitudes por segundo por método.<sup>21</sup>
  - *Manejo de Errores:* El sistema debe retornar el código HTTP 429 Too Many Requests cuando se excede el límite.
  - *Transparencia (Headers):* Es obligatorio para una buena experiencia de desarrollador (DX) incluir headers que informen sobre el estado del límite:
    - Retry-After: Segundos a esperar antes de reintentar.
    - X-RateLimit-Limit: Límite total permitido.
    - X-RateLimit-Remaining: Solicitudes restantes en la ventana actual.
  - *Implicación para Beta CRM:* Debemos implementar un middleware de Rate Limiting de alto rendimiento (e.g., basado en Redis con algoritmo "Token Bucket" o "Leaky Bucket") que intercepte las llamadas antes de que toquen la base de datos o los controladores lógicos.

### 5.2. Análisis de Endpoints Críticos y Payloads

Para replicar la funcionalidad, debemos clonar las capacidades de los endpoints más utilizados. A continuación se detalla la lógica de los endpoints esenciales de Mensajería y Contactos.

#### 1. Envío de Mensajes (POST /contact/{identifier}/message) 6:

La elegancia de este endpoint radica en la flexibilidad del parámetro {identifier} en la URL.

- *Enrutamiento Inteligente:* El desarrollador no necesita saber el ID interno del contacto. Puede usar id:123, email:cliente@gmail.com, o phone:+34600.... El sistema resuelve la identidad internamente.
- *Selección de Canal (Channel Routing):* Si el payload no especifica un channelId, el sistema enruta inteligentemente el mensaje al "Last Interacted Channel" (último canal de

interacción).<sup>6</sup> Esta lógica reduce masivamente la complejidad para el desarrollador externo.

- **Tipos de Mensaje:** Soporta texto plano y adjuntos. Sin embargo, para payloads complejos específicos de plataforma (como Carruseles de Facebook o Botones de Telegram), se requiere especificar el channelId explícito, ya que estos formatos no son universales.

## 2. Gestión de Contactos (POST /contact y GET /contact/list) 4:

El payload de creación es rico y permite poblar el CRM en una sola llamada.

- **custom\_fields:** Acepta un objeto para poblar campos definidos por el usuario de una vez.
- **tags:** Array de strings para segmentación inmediata (e.g., ["lead\_caliente", "origen\_web"]).
- **assignee:** Permite asignar el contacto a un agente específico (assignee\_id) o por email directamente en el momento de la creación, facilitando la integración con herramientas de distribución de leads externas.
- **Paginación:** El endpoint de listado utiliza paginación basada en cursor (cursorId, limit). Para la 'Beta CRM', se desaconseja la paginación por offset (página 1, 2, 3) ya que es ineficiente en grandes volúmenes de datos (Big Data); el cursor es el estándar moderno.

## 5.3. Webhooks: Ingesta de Eventos y Transformación de Datos

Respond.io utiliza Incoming Webhooks como disparadores de primera clase para sus flujos de trabajo.<sup>16</sup> La característica distintiva que debemos replicar es la flexibilidad en el manejo del payload.

- **Payload Agnóstico:** El sistema no impone una estructura rígida para los datos entrantes. Acepta cualquier JSON que envíe una aplicación tercera (Shopify, Typeform, servidor propio).
- **Mapeo de Variables (JSONPath):** Para hacer estos datos útiles, el sistema permite al usuario configurar mapeos visuales utilizando una sintaxis similar a JSONPath (e.g., \$.customer.email o \$.order\_id).<sup>16</sup>
- **Requisito para Beta CRM:** Debemos construir un parser que permita a los usuarios "explorar" el JSON de un webhook de prueba y seleccionar con clic qué campos desean guardar en variables internas del flujo o en campos del contacto, sin necesidad de escribir código.

Flujo de Procesamiento de API de Mensajería (Descripción Lógica):

El ciclo de vida de una petición API para enviar un mensaje sigue estos pasos rigurosos que el backend debe implementar:

1. **Recepción:** La petición llega al API Gateway.
2. **Autenticación y Rate Limit:** Se valida el Token Bearer y se consulta a Redis si el cliente tiene cupo en su bucket de peticiones. Si no, se devuelve 429 inmediatamente.
3. **Resolución de Identidad:** El sistema busca al contacto usando el identifier provisto (email, teléfono o ID). Si no existe, devuelve 404.
4. **Selección de Canal:** Si no se especifica canal, el sistema consulta la tabla

last\_interaction para determinar por dónde habló el usuario la última vez (e.g., WhatsApp).

5. **Validación de Ventana:** Se verifica si la ventana de 24h de dicho canal está abierta. Si está cerrada y el mensaje no es una plantilla, se devuelve error 400 o se falla la entrega.
6. **Despacho (Egress):** El mensaje se encola para ser enviado a la API del proveedor (Meta/Twilio/Telegram).

---

## 6. Recomendaciones Estratégicas para la Implementación de 'Beta CRM'

Basado en la investigación profunda de Respond.io, se presentan las siguientes recomendaciones arquitectónicas finales para el equipo de desarrollo de la 'Beta CRM':

1. **Priorizar la "Identidad Flexible":** No utilice nunca el número de teléfono o el email como clave primaria (PRIMARY KEY) en su tabla de contactos. Utilice un UUID interno agnóstico y mantenga tablas de índice secundario o de búsqueda para identity\_identifiers (email, phone, social\_ids). Esto es la única manera de soportar el "Identity Merging" sin dolorosas migraciones de datos futuras.
2. **Arquitectura de Workflows como Máquina de Estados:** No codifique la lógica de negocio (como el auto-cierre) en el código duro de la aplicación ("Hardcoded"). Construya o integre un motor de flujos ligero (BPMN-like) que permita definir estados (Wait) y transiciones (Close). Esto otorga a los equipos de operaciones el poder de iterar sus procesos sin depender de despliegues de ingeniería.
3. **Seguridad de IA en Capas:** Implemente la capa de "Boundaries" fuera del contexto del LLM. No confíe solo en que el prompt del sistema detendrá al modelo. Utilice validadores de salida deterministas (Regex) o clasificadores de texto pequeños y rápidos (BERT) para bloquear respuestas que contengan patrones prohibidos (e.g., números de tarjeta de crédito, promesas de precios no catalogados) antes de que lleguen al usuario.
4. **API Gateway Inteligente:** El router de canales (Last Interacted Channel) es una funcionalidad crítica para la facilidad de uso. Implemente una tabla ligera last\_interaction que guarde la tupla {contact\_id, channel\_id, timestamp} y actualicela con cada mensaje entrante. Úsela como el destino por defecto en su API de salida.
5. **Manejo de Webhooks No-Code:** Invierta tiempo en el UI/UX del mapeo de webhooks. La capacidad de ingerir datos de cualquier fuente sin middleware intermedio es un diferenciador clave de producto.

## Conclusión

Respond.io v2.0 demuestra una madurez significativa en la abstracción de la complejidad multicanal, ocultando la fricción de APIs dispares (Meta, Telegram, Email) bajo una interfaz unificada y lógica. Para replicar su éxito en una 'Beta CRM', el enfoque no debe ser cosmético; debe ser estructural. La robustez del backend en el enrutamiento de mensajes, la resolución



determinista de identidades conflictivas y la orquestación segura y restringida de agentes de IA son los pilares no negociables sobre los que se debe construir el sistema.

## Fuentes citadas

1. How to Write Effective AI Agent Prompts - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/ai-agents/how-to-write-effective-ai-agent-prompts>
2. Getting Started with AI Agents - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/ai-agents/getting-started-with-ai-agents>
3. Using AI Agent Actions - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/ai-agents/using-ai-agent-actions>
4. List the contacts | API v2 Documentation, acceso: diciembre 28, 2025, <https://developers.respond.io/docs/api/0759d02787ab3-list-the-contacts>
5. Get a contact | API v2 Documentation, acceso: diciembre 28, 2025, <https://developers.respond.io/docs/api/cbcfb23486778-get-a-contact>
6. Send a message | API v2 Documentation, acceso: diciembre 28, 2025, <https://developers.respond.io/docs/api/a748f5bfb1bb5-send-a-message>
7. Contacts Overview - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/contacts/contacts-overview>
8. Contact Details - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/contacts/contact-details>
9. How To Merge Contacts: A Complete Guide - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/blog/merge-contacts>
10. Facebook Private Replies - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/facebook-messenger/private-replies>
11. Salesforce - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/integrations/salesforce-integration>
12. HubSpot - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/integrations/hub-spot>
13. CRM Integration: Salesforce - Zapier - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/zapier/salesforce>
14. Step: Close Conversation - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/workflows/step-close-conversation>
15. Step: Open Conversation - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/workflows/step-open-conversation>
16. Workflow Triggers - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/workflows/workflow-triggers>
17. Workflow Steps Overview - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/workflows/workflow-steps>
18. Assignment and Closing a Conversation - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/help/inbox/assignment-and-closing-a-conversation>
19. Conversational Support: How it Improves Customer Experience - Respond.io, acceso: diciembre 28, 2025, <https://respond.io/blog/conversational-support>
20. Messaging | API v2 Documentation, acceso: diciembre 28, 2025, <https://developers.respond.io/docs/api/42fc125b57906-messaging>

21. respond.io API Documentation, acceso: diciembre 28, 2025,  
<https://developers.respond.io/>
22. Step: HTTP Request - Respond.io, acceso: diciembre 28, 2025,  
<https://respond.io/help/workflows/step-http-request>